HealthNexus
Citizen module

Full-stack guide: every feature, its files, its key functions and how they talk to each other

Features covered: account, onboarding, smartwatch, scoring pipeline, dashboard, history, alerts and advisories, visits, Nira assistant, profile, settings

Generated 27 October 2026 from the live code. Every path:line link opens VS Code at that line.

Open any line by hand: Ctrl+P, paste the path and line, press Enter.

Contents

0. How to use this guide 3

1. Architecture and the citizen module map 4

2. Foundations every feature uses 6

3. Account: sign-up, email verification, sign-in, sessions 9

4. Onboarding: profile, health centre and consent 11

5. Smartwatch: how a reading enters the system 13

6. From reading to alert: the scoring pipeline 15

7. Dashboard 18

8. History 20

9. Alerts and advisories 21

10. Visits 23

11. Nira, the records assistant (citizen view) 25

12. Profile 27

13. Settings 29

14. How the features talk to each other 32

15. API index for the citizen module 34

16. Data model cheat-sheet 37

17. Numbers to know 38

18. Questions across the whole module 39

0 How to use this guide

Every function named here has a clickable link that opens VS Code at the exact line.

Opening any line the examiner asks for

- **Click the blue `path:line` link.** It uses the `vscode://file/…` address, so VS Code opens that file at that line. If your PDF viewer asks permission, allow it once.
- **Or type it.** In VS Code press `Ctrl+P`, paste `backend/app/services/readings.py:79` and press Enter. Every link is written in this form.
- **Line numbers are real.** They were looked up in the code when this PDF was generated. If the code changes later, regenerate with `python docs/citizen_guide/build.py` (needs Chrome or Edge and PyMuPDF); a stale reference stops the build instead of printing a wrong number.
- **If a link does nothing,** your PDF viewer is blocking the `vscode://` address. Use the Ctrl+P method above; the text of every link is exactly what to type.

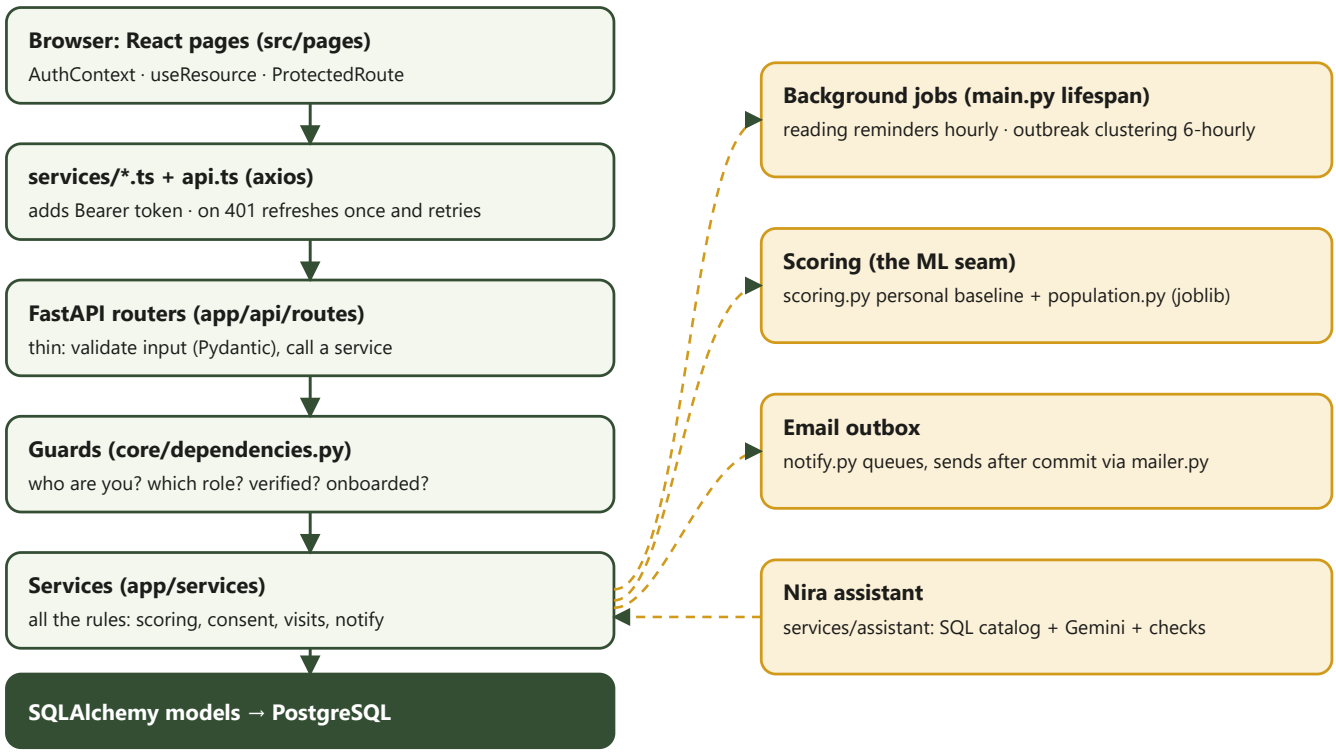
> Tip for the viva: say **which layer** first (page, API client, route, guard, service, table), then open the line. Every feature in this guide is walked in that order.

The layers, in one line each

Layer	Lives in	Rule of thumb
Page / component	`frontend/src/pages`, `components`	Shows data, collects input, calls one service function.
Service (frontend)	`frontend/src/services/*.ts`	One function per API call; no UI.
Route	`backend/app/api/routes`	Checks who is asking and validates input, then calls a service.
Guard	`backend/app/core/dependencies.py`	Refuses the wrong role or an unverified account before the route runs.
Service (backend)	`backend/app/services`	The rules: scoring, consent, visits, notifications.
Model	`backend/app/models`	Table definitions; the only code that knows column names.

1 Architecture and the citizen module map

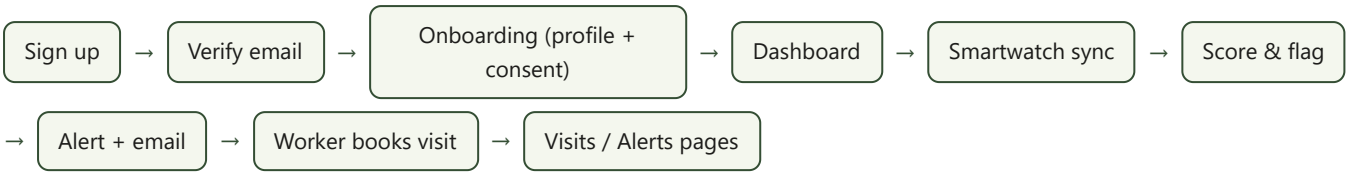
A browser app talks to one API; the API applies rules and stores everything in PostgreSQL.



Technology

Part	What is used	Where
UI	React 18, TypeScript, Vite, Tailwind CSS v4, react-router 7, Recharts (charts), Leaflet (maps)	`frontend/`
API client	axios instance with request and response interceptors	`src/services/api.ts:60`
API	FastAPI (async), Pydantic v2 validation	`app/main.py:40`
Storage	PostgreSQL through SQLAlchemy 2 (async, asyncpg); schema versions in Alembic (migrations 0001 to 0020)	`backend/app/models`, `backend/alembic/versions`
Passwords and tokens	bcrypt hashes; JWT (HS256) access tokens; random refresh tokens stored only as SHA-256 hashes	`app/core/security.py:29`
Anomaly scoring	Personal-baseline rules plus a trained Isolation Forest (scikit-learn) that only informs	`app/services/scoring.py:176`
Email	Outbox sent after the database commit; prints to the console unless SMTP is configured	`app/services/notify.py:35`
Assistant	Gemini picks one of 29 vetted SQL queries and words the answer; a template answers if Gemini fails	`backend/app/services/assistant`

The citizen module at a glance

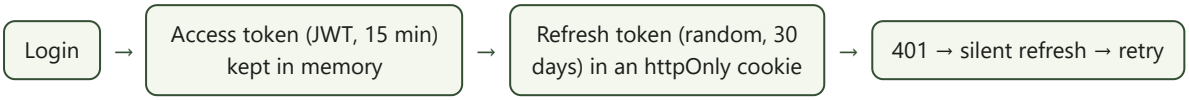


Feature	Page	Frontend service	Backend route	Rules in	Main tables
Account	SignupPage, LoginPage, VerifyEmail, SessionsPage	auth.ts	routes/auth.py	accounts.py, sessions.py	users, refresh_tokens
Onboarding	Onboarding.tsx	onboarding.ts	routes/onboarding.py	onboarding.py, consent.py	user_profiles, consent_records
Smartwatch	citizen/Watch.tsx	health.ts	routes/vitals.py (/health/watch)	watch.py, readings.py	health_records, processed_data
Dashboard	citizen/Dashboard.tsx	health.ts	routes/vitals.py (/health/summary)	scoring.py, readings.py	health_records, alerts, scheduled_visits
History	citizen/History.tsx	health.ts	routes/vitals.py (/health/history)	triage.py	health_records, processed_data
Alerts	citizen/Alerts.tsx	health.ts	routes/alerts.py	visits.py, notify.py	alerts, advisories
Visits	citizen/Visits.tsx	health.ts	routes/schedule.py	visits.py, visit_state.py	scheduled_visits
Nira	Assistant.tsx	assistant.ts	routes/assistant.py	services/assistant/*	chat_sessions, chat_messages
Profile	Profile.tsx	account.ts	routes/profile.py	(in the route)	users
Settings	Settings.tsx	account.ts	routes/settings.py	preferences.py, consent.py, complaints.py	users, consent_records, concerns

2 Foundations every feature uses

Sign-in state, guards, error handling and polling are written once and reused by every page.

2.1 The two tokens



- The **access token** is a signed JWT held only in a JavaScript variable (`accessToken` in `api.ts`). A page reload loses it on purpose; `AuthProvider` gets a new one from the refresh cookie at start-up.
- The **refresh token** is random, stored in the database only as a SHA-256 hash, sent as an `httpOnly` cookie limited to `/api/v1/auth`. JavaScript cannot read it, so a script injected into the page cannot steal it.
- Every refresh **rotates** the token. Presenting an old one again is treated as theft: the whole chain is revoked (`_revoke_chain`). A 10-second grace covers two tabs refreshing together.
- Each refresh token is valid for 30 days from when it was issued, and every refresh issues a new one, so an active person stays signed in. A session with **no refresh for 48 hours** is ended as idle (`REFRESH_IDLE_TIMEOUT`).

Function / component	Where	What it does
`create_access_token / decode_access_token`	app/core/security.py:66	Signs and checks the 15-minute JWT (claims: user id, role, session id).
`hash_password / verify_password`	app/core/security.py:29	bcrypt; passwords over 72 bytes are refused rather than silently cut.
`get_current_user`	app/core/dependencies.py:28	Decodes the token, then **reloads the user from the database**, so a deactivation or approval applies at once instead of after 15 minutes.
`rotate_session`	app/services/sessions.py:99	Validates a refresh token, revokes it, issues the next one; handles reuse, expiry and idle timeout.
`_revoke_chain`	app/services/sessions.py:88	Revokes every live descendant of a reused token (the thief's copy).
`request interceptor`	src/services/api.ts:65	Adds `Authorization: Bearer …` to every request.
`refreshSession`	src/services/api.ts:79	Single-flight refresh: concurrent callers share one request.
`response interceptor`	src/services/api.ts:113	On a 401 it refreshes once and replays the request; login and refresh are never retried.
`AuthProvider`	src/context/AuthContext.tsx:20	Holds `user` and `status`; restores the session on load; exposes login, logout, refreshUser.

2.2 Who may open what

Function / component	Where	What it does
`require_user`	app/core/dependencies.py:51	Citizen accounts only.
`require_onboarding_user`	app/core/dependencies.py:70	A citizen **with a verified email**. Used by the health, schedule and watch routes.

Function / component	Where	What it does
`require_any_verified`	app/core/dependencies.py:77	Any approved role with a verified email (settings, profile, alerts, assistant).
`ProtectedRoute`	src/components/ProtectedRoute.tsx:13	Signed out → login; wrong role → unauthorized; unverified → verify page; no profile yet → onboarding.
`destinationFor`	src/lib/routes.ts:46	The one place that decides where a user belongs after login.
`route table`	src/main.tsx:102	Citizen pages are wrapped in `ProtectedRoute` for role `user`.

> The page guard is for **good behaviour**; the API guard is for **security**. A user who edits the front end still gets a 403 from the guard on the server.

2.3 Fetching, errors, polling

Function / component	Where	What it does
`useResource`	src/lib/useResource.ts:12	Loads data when its key changes; aborts the old request so a slow earlier response can never overwrite a newer one; returns `data, error, loading, reload`.
`normaliseDetail / ApiError`	src/services/api.ts:42	FastAPI errors can be a string or a list; this turns both into one message plus per-field errors.
`useBadges / loadBadges`	src/components/app/AppShell.tsx:99	Polls unread counts every 30 s; when the count grows it shows a toast and plays the alert sound.
`ToastProvider / play`	src/components/app/Toaster.tsx:30	One place for pop-up messages; each tone has a sound in `lib/sound.ts`.
`fmtStamp`	src/lib/format.ts:13	The single date-and-time format used on every page (India time).
`useT / useLabel`	src/i18n/index.ts:64	English, Tamil and Hindi text.
`lifespan`	app/main.py:18	Checks the JWT secret, starts the reminder and outbreak jobs, drains the email outbox on shutdown.
`get_session`	app/db/session.py:11	One database session per request.

Questions you may be asked

Q. Why keep the access token in memory and not in localStorage?
Anything in localStorage can be read by any script on the page. A memory variable disappears on reload, and the refresh cookie (httpOnly, path-limited) brings it back.

Q. What if two tabs refresh at the same moment?
The browser side shares one request (`refreshSession`). On the server a token used within 10 seconds of rotation is answered with 'already used' without revoking anything, and the client retries after 400 ms.

Q. Why reload the user on every request instead of trusting the role in the token?
So that approving, deactivating or changing a role takes effect immediately (see the comment in `get_current_user`).

3 Account: sign-up, email verification, sign-in, sessions

How a person becomes a verified, signed-in citizen, and how a session is protected.

File	Role
src/pages/SignupPage.tsx:1	Page shell; the form is `components/auth/SignupForm.tsx`.
src/components/auth/SignupForm.tsx:39	Field rules, citizen vs worker sign-up, then logs in after registering.
src/components/auth/LoginForm.tsx:21	Email and password; goes to `destinationFor(user)`.
src/pages/VerifyEmail.tsx:16	Redeems the emailed token once and lets the user resend (60 s apart).
src/pages/ForgotPassword.tsx:14	Asks for a reset link; the answer is the same whether or not the email exists.
src/pages/ResetPassword.tsx:14	Sets a new password from the emailed token.
src/pages/account/SessionsPage.tsx:17	Lists devices signed in; sign one or all others out.
src/services/auth.ts:78	One function per auth endpoint.
app/api/routes/auth.py:39	Register, login, refresh, logout, verify, reset, sessions.
app/services/accounts.py:94	Create users, check passwords, issue and redeem one-time tokens.
app/services/sessions.py:81	Refresh-token rows: start, rotate, revoke, list.
app/services/mailer.py:62	Builds and sends the verification and reset emails.

Sign-up to first sign-in, end to end

#	Layer	What happens	Open in code
1	Browser	The form checks the rules locally (the same rules the API enforces) and calls `registerCitizen`.	src/components/auth/SignupForm.tsx:106
2	API client	`POST /auth/register/citizen` with name, email, password.	src/services/auth.ts:78
3	Router	Validates the body: password 8 to 72 bytes, email normalised to lower case.	app/api/routes/auth.py:82
4	Service	Creates the user (bcrypt hash, role `user`), makes a verification token and stores **only its hash** with a 24-hour expiry.	app/services/accounts.py:86
5	Email	A background task emails a link to `/verify-email?token=…`.	app/services/mailer.py:62
6	Browser	The form then signs the new user in so they land on the verify page.	src/components/auth/SignupForm.tsx:115
7	Router	`POST /auth/login`: `authenticate`, then `start_session`, then the refresh cookie is set and the access token returned.	app/api/routes/auth.py:100
8	Service	`authenticate` gives the same error for an unknown email, a wrong password and a disabled account, and burns a hash check so timing does not reveal which.	app/services/accounts.py:94

#	Layer	What happens	Open in code
9	Browser	`ProtectedRoute` sends an unverified user to the verify page.	`src/components/ProtectedRoute.tsx:21`
10	Browser	The emailed link opens the page, which removes the token from the address bar and redeems it once.	`src/pages/VerifyEmail.tsx:30`
11	Service	`redeem_verification` marks the account verified and clears the token.	`app/services/accounts.py:122`
12	Browser	Verified but `profile_completed` is false, so the user is sent to onboarding (chapter 4).	`src/lib/routes.ts:56`

Other flows in this feature

Function / component	Where	What it does
`prepare_password_reset / redeem_password_reset`	`app/services/accounts.py:137`	Reset link valid for 1 hour and asked for at most once a minute; redeeming it signs out **every** session.
`list_sessions / revoke_session`	`app/api/routes/auth.py:157`	Backs the Sessions page; the current device is marked.
`device_label / client_ip`	`app/services/sessions.py:46`	Turns the browser's user agent into 'Chrome on Windows' for the sessions list.
`update_consent`	`app/api/routes/auth.py:254`	`PATCH /auth/consent`: a citizen changes consent; runs the same `change_consents` as Settings.

Questions you may be asked

Q. How do you stop someone discovering which emails have accounts?
Login returns one message for every failure, forgot-password answers the same way whether or not the address exists, and a dummy hash check keeps response time similar.

Q. Where is the verification token stored?
Only its SHA-256 hash is in the database (`hash_token`); the raw value exists only in the email link, so a database leak does not give working links.

Q. What happens if the user refreshes the verify page mid-way?
The token is read once into state and removed from the URL; a `useRef` guard stops React StrictMode from spending the single-use token twice.

4 Onboarding: profile, health centre and consent

A new citizen answers five short steps; the answers decide which health centre sees their readings.

File	Role
src/pages/Onboarding.tsx:90	The five-step wizard; the current step lives in the URL (`?step=`).
src/components/onboarding/steps.tsx:23	One component per step: About you, Location, Medical, Device and contact, Consent.
src/lib/onboardingDraft.ts:49	Keeps answers in `sessionStorage` per user, so a reload does not lose them.
src/services/onboarding.ts:43	`getStates`, `getDistricts`, `getFacilities`, `createProfile`.
app/api/routes/onboarding.py:49	Creates the profile and records consent in one transaction.
app/services/onboarding.py:69	Checks the chosen centre serves the chosen district; copies fields onto the rows.
app/services/consent.py:24	Appends a row for each choice that differs from the current state (a first answer, even a refusal, is always recorded). Rows are never edited.

What each step collects

Step	Fields	Why it is asked
1 About you	age, gender, height, weight, blood group	Shown on the health worker's record of the citizen.
2 Location	state, district, health centre, pincode, area	Links the citizen to the centre whose workers review their readings; the centre must serve that district.
3 Medical history	conditions, medicines (tags)	Gives a flag context for the worker.
4 Device and contact	smartwatch yes/no and brand, emergency contact	Decides whether the Smartwatch page starts connected.
5 Consent	health tracking, health-worker access, community surveillance	Each is recorded with time, source and IP; all can be changed later in Settings.

#	Layer	What happens	Open in code
1	Browser	Each step is validated with the same rules as the API; the user cannot jump past an invalid step (`firstInvalidStep`).	src/pages/Onboarding.tsx:53
2	Browser	The location step loads states, districts and centres in turn from the registry.	src/components/onboarding/steps.tsx:72
3	Router	The three list endpoints read the registry (`valid_phc_facilities`).	app/api/routes/onboarding.py:37
4	Browser	On the last step `createProfile` sends everything in one request.	src/pages/Onboarding.tsx:127
5	Service	`resolve_facility` rejects a centre that does not serve the district.	app/services/onboarding.py:69

#	Layer	What happens	Open in code
6	Router	One transaction creates the profile, links the centre, sets `profile_completed`, mirrors health-tracking consent onto the account and writes consent rows. A double submit hits a unique constraint and returns 409.	app/api/routes/onboarding.py:62
7	Browser	The draft is cleared, `refreshUser` reloads the account, and the citizen lands on the dashboard.	src/pages/Onboarding.tsx:137

Questions you may be asked

Q. Why one transaction?

Profile, centre link, completed flag and consent rows must exist together or not at all; a half-finished onboarding would leave an account that can neither log readings nor retry.

Q. What if the second tab submits too?

The unique `user_id` on the profile raises `IntegrityError`; the route turns it into a 409 and the page treats that as 'already saved'.

Q. Where is the 'consent' stored and can it be changed?

Append-only `consent_records` with time, source and IP; the latest row per type wins (`current_consents`). Settings → Privacy adds new rows, so the history of choices is kept.

5 Smartwatch: how a reading enters the system

Citizens do not type vitals. They connect a watch and fetch one reading per part of the day; this build simulates the watch.

File	Role
src/pages/citizen/Watch.tsx:26	Connect screen, four daily slots, the latest result and a disconnect button.
src/services/health.ts:205	`getWatch`, `connectWatch`, `disconnectWatch`, `syncWatch`.
app/api/routes/vitals.py:161	The four `/health/watch*` endpoints.
app/services/watch.py:53	Slots, the once-per-slot rule, and the simulator.
app/services/readings.py:145	Stores and scores every reading from any source (chapter 6).
app/models/health_record.py:25	A reading's `source` is watch, phc (examination) or self.

The four slots

Slot	Local hours	Note
Morning	05:00 to 12:00	
Afternoon	12:00 to 17:00	
Evening	17:00 to 21:00	
Night	21:00 to 05:00	00:00 to 05:00 counts as the **previous day's** night (`slot_at`).

#	Layer	What happens	Open in code
1	Browser	The page loads the state and the vitals vocabulary together.	src/pages/citizen/Watch.tsx:27
2	Router	`GET /health/watch` returns whether a watch is connected, the current slot and what each slot holds.	app/api/routes/vitals.py:138
3	Service	`slot_at(now)` picks the slot and the day it counts for; `synced_today` finds readings already stored.	app/services/watch.py:64
4	Browser	Not connected? The Connect component asks for a brand and posts it.	src/pages/citizen/Watch.tsx:160
5	Router	`connect_watch` sets `has_smartwatch` and `smartwatch_brand` on the profile.	app/api/routes/vitals.py:143
6	Browser	'Fetch from watch' calls `syncWatch`.	src/pages/citizen/Watch.tsx:44
7	Guard	`require_onboarding_user`: citizen, verified. Then the route refuses if health-tracking consent is off (403) or no watch is connected (409).	app/api/routes/vitals.py:165
8	Database	The user row is locked (`with_for_update`) so a double tap cannot create two readings.	app/api/routes/vitals.py:170
9	Service	If this slot already has a reading it is returned (`already_synced`); otherwise `simulate` draws one.	app/services/watch.py:73
10	Service	`submit_reading(source='watch', watch_day, watch_slot)` stores and scores it (chapter 6) and returns the result.	app/api/routes/vitals.py:179

#	Layer	What happens	Open in code
11	Browser	The result card appears; a flagged reading plays the alert sound, a normal one the clear sound.	src/pages/citizen/Watch.tsx:46

Why the simulator is believable: values are drawn around the person's own recent median (or typical adult values for a new person), shifted for time of day (lower heart rate at night, slightly warmer in the afternoon), and the random generator is seeded with `user-day-slot`, so the same slot always gives the same reading.

Questions you may be asked

Q. Why is the watch simulated?

A real wearable needs a vendor account and device. The app's contract is a reading with four vitals per slot; replacing `simulate` with a vendor API call changes one function and nothing else.

Q. How do you prevent two readings for one slot?

Three layers: the row lock in `sync_watch`, the check `synced_today`, and a partial unique index `uq_health_records_watch_slot` on (user, day, slot) for watch rows (migration 0015).

Q. Where do blood pressure and sugar come from?

A health worker records them during an examination (`source = phc`); they appear in History with the worker's name and place.

6 From reading to alert: the scoring pipeline

Every reading, from any source, goes through one function. This is the engine behind the dashboard, history, alerts and the worker's queue.

submit_reading → record_reading → earlier_vitals (last 120) → score_reading → population.check → ProcessedData saved → Alert row (if flagged) → notify.reading_flagged → email after commit

File	Role
app/services/readings.py:78	Stores the reading, scores it, writes the alert.
app/services/scoring.py:176	The only function the rest of the app calls to judge a reading.
app/services/population.py:69	Isolation Forest percentile for heart rate and breathing; informs, never flags.
app/services/triage.py:21	Score → critical / high / moderate / low, and the plain sentence for a flag.
app/services/metrics.py:54	The vitals vocabulary: normal bands, units, plausible ranges.
app/services/notify.py:65	Emails the citizen and (if reviewed) the centre's workers.
app/api/views.py:78	Shapes the result the page shows, including the per-vital explanation.

How a reading is judged

- **New person (fewer than 7 earlier readings of a vital):** compared with the published normal range only. Distance outside the range is measured in half-band widths (`published_contribution`).
- **Building a baseline (7 to 20 readings):** the weight moves gradually from the published range to the person's own median and robust spread (`baseline_weight`, `robust_baseline` using the median absolute deviation).
- **Personal (21 or more):** judged against the person alone. Someone whose blood pressure is always a little high is not flagged on every reading.
- **Confirmation rule:** a value inside the published range that is only unusual for this person needs a **second** unusual reading in the same direction before it counts (`_judge`). One odd home reading is usually noise.
- **Safety limits** (for example SpO_2 at or below 92, temperature at or above 38.0 °C) always count in full, whatever the person's usual (`past_safety_limit`).
- A vital counts only from a contribution of **0.5** (`FLAG_THRESHOLD`). `anomaly_score = min(1, strongest contribution / 4)`.

Severity	Score	Alert colour for the citizen
Critical	0.75 and above	High (red)
High	0.50 to 0.75	High (red)
Moderate	0.25 to 0.50	Medium (amber)
Low	below 0.25	Low (green)

Function / component	Where	What it does
`record_reading`	app/services/readings.py:78	Saves `HealthRecord`, scores it against the earlier readings, saves `ProcessedData` (features, per-vital details, population view) and the alert.
`earlier_vitals`	app/services/readings.py:51	Up to 120 earlier readings, oldest first, for the baseline.
`submit_reading`	app/services/readings.py:145	record + queue emails + commit. Every way a reading arrives goes through here.
`_citizen_message`	app/services/readings.py:62	Tells the citizen honestly whether anyone is reviewing the flag (no worker, or worker access switched off).
`_judge`	app/services/scoring.py:136	One vital: published contribution, personal contribution, weight, safety and confirmation.
`_contribution`	app/services/scoring.py:166	Blends published and personal parts; applies the 0.5 threshold and safety limits.
`describe_signal`	app/services/triage.py:65	Builds 'Oxygen saturation 89% — well below the normal range' from the strongest feature.
`alert_severity`	app/services/triage.py:35	Four severities map to the alerts table's three levels.
`check (population)`	app/services/population.py:69	Scores heart rate and breathing against 71 people's data; stored for the worker, never decides a flag.

#	Layer	What happens	Open in code
1	Service	Earlier readings are loaded so the baseline never includes the reading being judged.	app/services/readings.py:93
2	Database	The `HealthRecord` is stored with the citizen's location.	app/services/readings.py:94
3	Model	`score_reading` judges each vital and returns features, score, basis and details.	app/services/readings.py:112
4	Database	`ProcessedData` keeps the working so a flag can explain itself later.	app/services/readings.py:113
5	Service	A flagged reading creates the citizen's `Alert`.	app/services/readings.py:132
6	Email	`reading_flagged` queues emails according to each person's preferences and minimum severity.	app/services/notify.py:65
7	Database	One commit; queued emails are sent only after it succeeds and dropped if it rolls back.	app/services/notify.py:45
8	Browser	The 30-second badge poll notices the new unread alert and shows a toast with the alert sound.	src/components/app/AppShell.tsx:113

Questions you may be asked

Q. Is it machine learning?

The decision is a transparent statistical scorer (personal median and spread), chosen because it can explain every flag. A trained Isolation Forest also scores each reading, but it only adds a 'population check' because on held-out people it caught nothing the rules missed.

Q. Why can a flag be explained to a citizen?

`ProcessedData.details` stores, for each vital, the value, the baseline, the spread, the z-score, the weight and whether it was confirmed; the page renders it as sentences.

Q. What does the citizen read when nobody is reviewing?

`_citizen_message` says so plainly and advises seeing a doctor, instead of the reassuring 'your centre has been told'.

7 Dashboard

One call returns everything the home page needs: latest values, trends, flags, alerts, next visit and centre.

File	Role
src/pages/citizen/Dashboard.tsx:265	The page: score ring, vital chart, time-of-day and week comparison, latest alerts.
src/services/health.ts:219	`GET /health/summary`.
app/api/routes/vitals.py:311	Builds the summary from `health_records`, `processed_data`, `alerts`, `scheduled_visits`.
app/services/readings.py:42	The 'usual' range shown next to each latest value.
src/components/app/TrendChart.tsx:1	Recharts line chart: the shaded band is the published normal range and flagged readings are drawn larger.

Function / component	Where	What it does
`summary`	app/api/routes/vitals.py:311	One request: newest value of each vital with its personal baseline, 30-day trend, counts, five unread alerts, next visit, centre and how many workers serve it.
`factorsOf`	src/pages/citizen/Dashboard.tsx:42	Builds the health score: vitals with a normal band share 80 points; open flags cost up to 20.
`HealthRing`	src/pages/citizen/Dashboard.tsx:81	The ring: one arc per factor; hover shows why; click jumps to that vital's chart.
`TimeOfDay`	src/pages/citizen/Dashboard.tsx:161	Average of the chosen vital by morning, afternoon, evening and night.
`WeekComparison`	src/pages/citizen/Dashboard.tsx:216	This week against the week before.
`CitizenDashboard`	src/pages/citizen/Dashboard.tsx:286	Shows a red notice when no centre or no worker is assigned: a flag would go unreviewed.

How the health score is made (100 points)

Part	Points	Rule
Each vital with a normal range	80 shared equally	Inside the band keeps all its share. Outside, it loses 80% of its share for every half-band width away from the band, down to zero.
Open flags	20	Each flag not yet reviewed by the centre costs 7 points, to a maximum of 20.
Verdict		85 and above 'Looking well'; 65 and above 'Worth watching'; below that 'Needs attention'.

> The score is a **display summary** computed in the browser from values the API already returned. It does not feed any alert, so it can be explained on the page (hover each arc) without exposing the model.

#	Layer	What happens	Open in code
1	Browser	`useResource` fetches the summary and the vitals vocabulary together.	src/pages/citizen/Dashboard.tsx:269
2	Guard	`require_onboarding_user`: only the signed-in, verified citizen; every query below filters on that user's id.	app/api/routes/vitals.py:311
3	Database	Readings of the last 90 days (capped at 500) joined to their scores.	app/api/routes/vitals.py:313
4	Service	For each vital the newest value is compared with that vital's earlier values (`baseline_of` needs 7).	app/api/routes/vitals.py:330
5	Database	Open flags, unread alerts, next scheduled visit and the centre are counted or fetched.	app/api/routes/vitals.py:367
6	Browser	`factorsOf` turns the latest values into score factors; `HealthRing` draws them.	src/pages/citizen/Dashboard.tsx:285

Questions you may be asked

Q. Where do the 'usual' ranges come from?

From the citizen's own earlier readings in the window (median and robust spread), shown only once there are at least 7; before that no baseline is shown rather than a fake one.

Q. What does the red notice mean?

The citizen has no centre, or their centre has no active worker, so a flagged reading has nobody to review it. It links to Settings → Your health centre.

8 History

Every reading the citizen has, filterable, with the model's explanation and a delete for their own entries.

File	Role
src/pages/citizen/History.tsx:17	Filters (measurement, period, flagged only), paged list, expandable explanation, delete with confirmation.
src/lib/params.ts:9	Filters live in the URL; `oneOf` and `intParam` clamp whatever is typed there before it reaches the API.
app/api/routes/vitals.py:273	`GET /health/history`: filters, pages of 20, outer join to the score.
app/api/routes/vitals.py:402	`DELETE /health/readings/{id}`.
app/services/triage.py:65	The one-line reason shown for a flagged reading.

#	Layer	What happens	Open in code
1	Browser	The URL holds the filters; they are validated against the allowed lists.	src/pages/citizen/History.tsx:22
2	API client	`getHistory` sends page, page size, metric, days, flagged-only.	src/services/health.ts:211
3	Router	Unknown metric names are rejected (422). The query always starts from `HealthRecord.user_id == user.id`.	app/api/routes/vitals.py:282
4	Database	Outer join to `processed_data`; ordered newest first; offset and limit for the page; total counted separately.	app/api/routes/vitals.py:286
5	Service	`_reading_out` adds severity, the signal sentence, who recorded it and the population view.	app/api/routes/vitals.py:208
6	Browser	Each row shows source (watch, examination), place and worker for examinations, and 'why it was flagged' on demand.	src/pages/citizen/History.tsx:44
7	Router	Delete is scoped to the caller, so someone else's id is simply 'not found'. A worker's examination cannot be deleted by the citizen (409).	app/api/routes/vitals.py:413
8	Database	The score is deleted with the reading and an `AuditLog` row 'delete_reading' is written, in one commit.	app/api/routes/vitals.py:420

Questions you may be asked

Q. Can a citizen delete a flagged reading to make a flag disappear from the worker's queue?

Yes for their own watch readings, and the deletion is audited. Readings recorded by a worker (`source = phc`) can only be changed by the centre.

Q. Why filter in the URL?

A refreshed or shared page keeps the same view, and clamping the parameters means nothing unexpected reaches the API.

9 Alerts and advisories

Two lists on one page: notices about the citizen's own readings and visits, and community advisories for their area.

File	Role
src/pages/citizen/Alerts.tsx:51	Tabs for Your alerts and Advisories; search and filters; detail pop-ups.
src/services/health.ts:226	`getAlerts`, `getUnreadCount`, `markRead`.
app/api/routes/alerts.py:30	Own alerts plus the advisories addressed to this citizen.
app/api/routes/alerts.py:62	Marks alerts read; scoped to the caller.
app/api/routes/advisories.py:68	Which advisories a person is shown.
src/components/app/AppShell.tsx:74	The sidebar badge: unread count, refreshed every 30 s.

#	Layer	What happens	Open in code
1	Browser	`getAlerts` returns `alerts`, `advisories` and the unread count.	src/pages/citizen/Alerts.tsx:54
2	Router	Own alerts: `Alert.user_id == user.id`, newest first.	app/api/routes/alerts.py:36
3	Service	`targeting(user)` matches an advisory to the citizen by centre, by a 'living nearby' radius, by locality or pincode, by district, or by state.	app/api/routes/advisories.py:68
4	Router	Advisories of the last 365 days are returned, running ones first, so the page can show history.	app/api/routes/alerts.py:43
5	Browser	`topic()` names each alert (Visit booked, Reading flagged, Smartwatch…) from its wording.	src/pages/citizen/Alerts.tsx:33
6	Browser	Opening an alert marks just that one as seen; 'Mark all as seen' clears the rest.	src/pages/citizen/Alerts.tsx:94
7	Router	`mark_read` has `user_id` in the WHERE, so ids belonging to anyone else match nothing.	app/api/routes/alerts.py:65
8	Browser	The sidebar badge drops on the next poll.	src/components/app/AppShell.tsx:47

Where alerts come from

Alert	Created by	Code
Reading flagged	Scoring pipeline	app/services/readings.py:132
Visit booked	Worker books a visit	app/services/visits.py:24
Visit moved or cancelled	Worker or citizen changes a visit	app/services/visits.py:40
Visit has no outcome	First schedule load after the visit time plus 4 hours	app/services/visits.py:56
Reading reminder	Hourly job: nothing logged for 3 days	app/services/reminders.py:36

Questions you may be asked

Q. Is an alert 'acted upon'?

On this page 'Seen' means the citizen opened it (it is marked read then). The health worker's action on the underlying flag (resolved or dismissed) is on the worker side and the citizen's History shows that it was reviewed.

Q. How is an advisory addressed to just one locality?

The admin's advisory stores a scope (centre, area, district, state); `targeting` compares it with the citizen's centre, locality, pincode, district and home coordinates.

10 Visits

The citizen sees the visits a health worker booked, can cancel with a reason up to 8 hours ahead, and can explain a missed one.

File	Role
src/pages/citizen/Visits.tsx:26	Tabs Upcoming, Done, Missed, Cancelled; search and filters; detail pop-up.
src/components/app/visits.tsx:17	Shared pieces: the 8-hour check, status pill, cancel form, modal.
app/api/routes/schedule.py:49	`GET /schedule/mine`.
app/api/routes/schedule.py:235	`POST /schedule/mine/{id}/cancel`.
app/api/routes/schedule.py:212	`POST /schedule/mine/{id}/reason`.
app/services/visit_state.py:42	Derives upcoming, today, due_now, overdue from the status and the clock.
app/services/visits.py:56	One-time 'no outcome' alerts, claimed atomically.
app/services/complaints.py:56	Counts unexplained misses for a worker's complaint.

#	Layer	What happens	Open in code
1	Browser	The page loads the citizen's visits.	src/pages/citizen/Visits.tsx:27
2	Router	`notify_missed` first writes any due 'no outcome' alerts; then visits of the last 180 days, or still scheduled, are returned with the worker's phone.	app/api/routes/schedule.py:52
3	Service	`visit_state` is derived at read time (4-hour grace), never stored.	app/services/visit_state.py:48
4	Browser	Cancel is offered only if the visit is more than 8 hours away (`changeable`) and a reason of 10 or more characters is typed.	src/components/app/visits.tsx:17
5	Router	The server enforces the same rule; a visit under 8 hours returns 409.	app/api/routes/schedule.py:36
6	Database	Status becomes cancelled; `cancelled_by = citizen`, the reason and time are stored.	app/api/routes/schedule.py:250
7	Service	`notify_changed` alerts the worker with the citizen's reason and queues the emails.	app/services/visits.py:40
8	Browser	A missed visit shows a box to say why; the reason is saved with `explainMissedVisit`.	src/pages/citizen/Visits.tsx:203
9	Service	An explained miss no longer counts toward a worker's complaint (needs 3 unexplained in 90 days).	app/services/complaints.py:22

> Visits are **created by the health worker** (from the worker's Alerts, Citizens or Schedule pages). The citizen's page is read, cancel and explain. Booking creates the citizen's alert and email through `notify_booked`.

Questions you may be asked

Q. Why 8 hours?

Late cancellations leave a worker with a wasted trip. The rule is one constant (`CHANGE_CUTOFF`) enforced on the server and mirrored in the page so the button hides when it cannot work.

Q. What if the citizen withdraws health-worker access or changes centre?

`change_consents` and `change_health_centre` call `cancel_open_visits`, because those workers can no longer see this person (chapter 13).

11 Nira, the records assistant (citizen view)

A chat that answers from the citizen's own records. It retrieves with fixed SQL, then words the answer, then checks the numbers.

Question → Safety check → Plan: pick queries → Retrieve (read-only SQL) → Word the answer → Check every number → Store with its sources

File	Role
src/pages/Assistant.tsx:100	Chat list, greeting cards, composer, answer bubbles with a 'Based on' source line.
src/services/assistant.ts:47	`POST /assistant/ask` and the session calls.
app/api/routes/assistant.py:154	Rate limit (20 a minute), chat ownership, then `assistant.ask`.
app/services/assistant/__init__.py:217	The conductor: safety, plan, retrieve, write, verify.
app/services/assistant/router.py:24	Emergency and medical-advice rules, and the keyword planner used when Gemini is unavailable.
app/services/assistant/catalog.py:740	The vetted SQL queries; 11 of the 29 are open to citizens.
app/services/assistant/llm.py:104	Gemini function calling: chooses up to three queries; `write_answer` words the result.
app/services/assistant/grounding.py:56	Finds numbers in an answer that are not in the retrieved rows.
app/services/assistant/compose.py:68	Template answers, used when Gemini fails or its answer is rejected.

#	Layer	What happens	Open in code
1	Browser	`send` posts the question with the current chat id.	src/pages/Assistant.tsx:182
2	Router	Rate limit, then the chat is loaded and checked to belong to this user.	app/api/routes/assistant.py:157
3	Service	Emergency words get 'call 108' at once; diagnosis or medicine questions are declined.	app/services/assistant/__init__.py:220
4	Service	Gemini (or the keyword router) picks which queries answer it, with cleaned parameters.	app/services/assistant/__init__.py:184
5	Database	Each query runs in a **read-only** transaction with a timeout; `:me` is bound from the signed-in user, never from the question.	app/services/assistant/__init__.py:141
6	Model	Gemini writes the answer from those rows only.	app/services/assistant/llm.py:133
7	Service	Every number in the answer must appear in the rows; otherwise the template answer is used.	app/services/assistant/__init__.py:276
8	Database	The question, answer, the sources and whether the numbers were checked are stored.	app/services/assistant/__init__.py:333
9	Browser	The bubble shows 'Based on: …' and a 'Numbers checked' badge; the rows are one click away.	src/pages/Assistant.tsx:604

Questions you may be asked

Q. Is this RAG?

Yes: it retrieves private data at question time, adds it to the prompt and generates from it. The retriever is parameterised SQL instead of a vector search because the data is structured and needs exact counts, averages and access control.

Q. Can the assistant read someone else's records?

No. The scope (`:me`) is bound on the server from the signed-in user, the SQL is a fixed catalog the model cannot edit, and the transaction is read-only.

12 Profile

Name, contact details, photo and cover picture, and a safe way to change the sign-in email.

File	Role
src/pages/Profile.tsx:47	Edit form, avatar and cover pickers, change-email form.
src/pages/ConfirmEmail.tsx:7	Opened from the link in the email to the new address.
src/services/account.ts:120	`getProfile`, `updateProfile`, `uploadImage`, `removeImage`, `requestEmailChange`, `confirmEmailChange`.
app/api/routes/profile.py:121	`GET/PATCH /profile`, images, email change.
app/models/user.py:35	The columns edited here: name, phone, bio, location, date_of_birth, avatar_url, cover_url.

Function / component	Where	What it does
`ProfilePatch`	app/api/routes/profile.py:75	Allowed fields only (`extra=forbid`); phone and date of birth are validated; bio at most 280 characters.
`update_profile`	app/api/routes/profile.py:121	Applies only the fields that were sent; a name cannot be set empty.
`_store`	app/api/routes/profile.py:157	Reads at most limit+1 bytes (2 MB avatar, 5 MB cover), checks the **file's own bytes** (`_sniff`), stores under an unguessable name and removes the old file.
`_sniff`	app/api/routes/profile.py:137	Recognises PNG, JPEG and WebP from their first bytes, not from the name or the browser's content type.
`_remove_old`	app/api/routes/profile.py:146	Deletes only inside our own media folder; refuses path tricks.
`request_email_change`	app/api/routes/profile.py:217	Needs the current password; stores `pending_email` and a hashed token; mails the **new** address.
`confirm_email_change`	app/api/routes/profile.py:243	Public (the link opens on any device); swaps the address; mails the **old** address a notice.
`ImagePicker`	src/pages/Profile.tsx:13	Checks the size in the browser first, then uploads.

Changing the email address, end to end

#	Layer	What happens	Open in code
1	Browser	The form sends the new address and the current password.	src/pages/Profile.tsx:98
2	Router	Wrong password, same address, address already taken and 'asked within a minute' are each refused.	app/api/routes/profile.py:223
3	Database	`pending_email` and the token **hash** with a 24-hour expiry are stored; the current address keeps working.	app/api/routes/profile.py:234
4	Email	The confirmation link goes to the new address only.	app/services/mailer.py:84
5	Browser	The link opens `/confirm-email?token=…`.	src/pages/ConfirmEmail.tsx:7

#	Layer	What happens	Open in code
6	Router	The token is checked, the address swapped, and a unique-constraint clash (someone registered it meanwhile) returns 409.	app/api/routes/profile.py:257
7	Email	The old address gets 'your email was changed' so a hijack is noticed.	app/services/mailer.py:95

Questions you may be asked

Q. Why check the image's bytes?

A file named `a.png` can be anything; content type headers are written by the sender. Checking the signature means only real PNG, JPEG or WebP files are stored.

Q. Why mail the old address after the change?

If someone with a stolen session changed the email, the owner finds out.

13 Settings

Five tabs. Each tab is one component in `Settings.tsx` and one group of endpoints in `settings.py`.

File	Role
src/pages/Settings.tsx:502	The tab bar; the active tab lives in `?tab=`.
src/services/account.ts:87	One function per settings call.
app/api/routes/settings.py:40	Preferences, consents, about, export, password, deactivate, complaints, health centre.
app/services/preferences.py:76	Defaults, validation and the helpers notifications read.
app/services/consent.py:87	Records a choice and carries out what it means.
app/services/complaints.py:119	Checks a complaint is allowed and attaches the app's evidence.

Tab	Component	Endpoints
Account & security	src/pages/Settings.tsx:36	POST /settings/password, POST /settings/deactivate (and the Sessions page)
Preferences	src/pages/Settings.tsx:111	GET/PATCH /settings/preferences
Privacy	src/pages/Settings.tsx:230	GET/PATCH /settings/consents, GET /settings/consents/history, GET /settings/about, GET /settings/export
Support	src/pages/Settings.tsx:306	POST/GET /settings/concerns
Your health centre	src/pages/Settings.tsx:375	GET/PATCH /settings/health-centre

13.1 Account and security

#	Layer	What happens	Open in code
1	Browser	Current and new password are sent.	src/pages/Settings.tsx:53
2	Router	The current password is verified; the new one must differ.	app/api/routes/settings.py:279
3	Service	The hash is replaced and **every other session** is revoked; this device stays signed in.	app/api/routes/settings.py:294
4	Router	Deactivate needs the password **and** the typed word DEACTIVATE; it clears the account's access, cancels open visits, revokes sessions and writes an audit row.	app/api/routes/settings.py:305
5	Guard	A deactivated user fails `get_current_user` on the very next request.	app/core/dependencies.py:41

13.2 Preferences

- Stored as one JSON column on the user (`users.preferences`): theme, language (English, Tamil, Hindi), whether alerts and visits are also emailed, the **least severe flag worth an email**, and reading reminders.
- `PATCH` accepts any subset, **deep-merges** it into the current values and validates the merged result as a whole (`update_preferences`), so a partial save can never leave an invalid state.

- Everything is always shown in the app; these settings only choose what is **also emailed** (`emails_about`, `emails_flag` in `preferences.py`).

Function / component	Where	What it does
`update_preferences`	app/api/routes/settings.py:53	Merge, validate, store.
`current_preferences`	app/services/preferences.py:76	Stored values over defaults; anything stored that no longer validates falls back to the default.
`emails_flag`	app/services/preferences.py:89	True only if flag emails are on **and** the flag is at least as severe as the person's minimum.
`PreferencesTab.save`	src/pages/Settings.tsx:121	Sends the changed field; applies theme and language immediately.

13.3 Privacy: what each consent does

Consent	Default if never answered	Effect when off
Health tracking	set at onboarding	Watch sync returns 403 and no new readings are stored; reminders stop. Mirrored onto `users.consent` and the profile.
Health worker access	**Yes**	The citizen disappears from the centre's lists (`allows_worker_access`); open visits are cancelled (`cancel_open_visits`).
Community monitoring	**No**	The citizen is left out of locality figures, the map and outbreak clustering (`counts_in_community`).
Research	**No**	Left out of the de-identified research export (`in_research`).

Function / component	Where	What it does
`change_consents`	app/services/consent.py:87	Appends the choice, mirrors health tracking, cancels visits if worker access is withdrawn. Caller commits.
`allows_worker_access` / `counts_in_community`	app/services/consent.py:58	SQL helpers every worker-side and area-level query includes.
`about`	app/api/routes/settings.py:141	Builds the plain-language 'who can see your data' sentence from the real consent state.
`export`	app/api/routes/settings.py:187	Downloads everything held about the citizen as one JSON file (profile, readings with scores, alerts, visits, consent history, complaints).
`PrivacyTab.change`	src/pages/Settings.tsx:244	Sends one change; reloads the list and the history.

13.4 Support (complaints)

- A citizen can raise: smartwatch not syncing, a worker did not come, a worker's behaviour, a wrong flag, data misuse, an access request, app problem, other.
- `complaints.prepare` **attaches evidence from the app itself**: for a device complaint the last sync time and the readings by source; for a no-show the visits whose time passed with no outcome recorded.
- The citizen's list shows each request with its reference, status (open, in review, resolved or rejected) and the admin's reply (`admin_note`).

Function / component	Where	What it does
`raise_concern`	app/api/routes/settings.py:349	Creates the complaint with `subject_id` and `evidence` from `prepare`.
`prepare`	app/services/complaints.py:119	Validates the category for the role and gathers evidence.
`_watch`	app/services/complaints.py:98	Evidence for a smartwatch complaint.
`SupportTab`	src/pages/Settings.tsx:306	Form and the citizen's own list.

13.5 Your health centre

#	Layer	What happens	Open in code
1	Browser	The tab lists active centres in the citizen's own district with how many workers each has.	src/pages/Settings.tsx:375
2	Router	`_health_centre` builds the list: active centres of the district, worker counts, which one is current.	app/api/routes/settings.py:403
3	Router	Changing to a centre outside the district is refused.	app/api/routes/settings.py:449
4	Service	Visits booked by the old centre's workers are cancelled, since those workers lose sight of the citizen.	app/api/routes/settings.py:455
5	Database	`user.phc_id` changes and an `AuditLog` row 'change_health_centre' is written.	app/api/routes/settings.py:457

Questions you may be asked

Q. What happens to readings already stored if the citizen changes centre?
They stay with the citizen. The new centre's workers see them (subject to consent) because access is by the user's current `phc_id`.

Q. Is anything deleted when an account is deactivated?
No data is deleted; the account is switched off, sessions revoked and open visits cancelled. The export lets the person take their data.

Q. Why is 'never answered' treated as yes for worker access but no for community monitoring?
Choosing a centre at onboarding is what links the citizen to it, so worker access is the default and consent is how they withdraw. Area figures and research are optional extras that must be opted into.

14 How the features talk to each other

Features never call each other's pages. They share **tables and services**: one feature writes a row, another reads it.

The shared spine

health_records + processed_data → alerts → scheduled_visits → consent_records → users.preferences

These five stores are the whole conversation between features. Each row below names the writer, the table, and who reads it.

When this happens	Written by	Stored in	Then these react
Citizen fetches a smartwatch reading	app/api/routes/vitals.py:161	health_records, processed_data	Dashboard score and charts · History list · worker's queue (if flagged) · Nira answers
A reading is flagged	app/services/readings.py:132	alerts; email outbox	Alerts page and badge · toast and sound within 30 s · emails to citizen and centre workers per their preferences
Worker resolves or dismisses a flag	app/api/routes/phc.py:336	processed_data.resolution	Dashboard open-flag count and score rise · History shows 'reviewed'
Worker books a visit	app/api/routes/phc.py:858	scheduled_visits; alerts; email	Visits page · Alerts page · dashboard 'next visit' · email to the citizen
Citizen cancels a visit	app/api/routes/schedule.py:235	scheduled_visits (cancelled_by, reason)	Worker's schedule card shows the reason · worker alert and email
Visit time passes with no outcome	app/services/visits.py:56	alerts (once); `missed_notified_at`	Both people alerted · citizen can explain · explained misses stop counting toward a complaint
Citizen turns off 'health worker access'	app/services/consent.py:87	consent_records	Hidden from every worker query · open visits cancelled · `about` text changes
Citizen turns off 'health tracking'	app/services/consent.py:104	consent_records, users.consent, profile	Watch sync returns 403 · reminders stop
Citizen turns on 'community monitoring'	app/services/consent.py:67	consent_records	Counted (without name) in locality figures, the risk map and outbreak clustering
Citizen changes centre	app/api/routes/settings.py:443	users.phc_id, audit_logs	Old centre's visits cancelled · new centre's workers see them · dashboard notice may clear
Citizen changes email or notification choices	app/api/routes/settings.py:53	users.preferences	`emails_about` / `emails_flag` decide every later email

When this happens	Written by	Stored in	Then these react
Quiet for 3 days	app/services/reminders.py:36	alerts; email	Reminder on the Alerts page and by email, once per quiet stretch, between 09:00 and 20:00
Admin publishes an advisory	app/api/routes/advisories.py:68	advisories	Appears on the matching citizens' Alerts page (Advisories tab)
Citizen raises a complaint	app/api/routes/settings.py:349	concerns (with evidence)	Admin's Complaints page; the reply returns to the citizen's Support tab

Three journeys end to end

A. A flagged reading reaches a person who can act

Watch sync → score_reading flags SpO_2 89 → ProcessedData + Alert → Emails queued → Commit →
Worker's Alerts card → Worker books a visit → Citizen toast + sound → Visits page

B. The citizen withdraws health-worker access

Settings → Privacy → PATCH /settings/consents → change_consents → ConsentRecord appended →
cancel_open_visits → Worker queries skip the citizen → Dashboard says nobody is reviewing

C. A visit the citizen cannot attend

Visit card (8+ hours away) → Cancel with reason → check_cutoff passes → status = cancelled →
Worker alert with reason → Slot freed on the worker's Schedule

> **If the examiner asks 'how do two features communicate?'** answer with this chapter's pattern: one feature writes a row through a service function; the other reads that table through its own route. There is no feature-to-feature call, which is why a page can be rewritten without touching another.

15 API index for the citizen module

Every endpoint a citizen's screens use, generated by scanning the route files, with the front-end function that calls it.

Endpoint	Handler	Called from
POST `/auth/register/citizen`	app/api/routes/auth.py:82	src/services/auth.ts:78
POST `/auth/login`	app/api/routes/auth.py:100	src/services/auth.ts:84
POST `/auth/refresh`	app/api/routes/auth.py:111	src/services/api.ts:84
POST `/auth/logout`	app/api/routes/auth.py:131	src/services/auth.ts:93
GET `/auth/me`	app/api/routes/auth.py:144	src/services/auth.ts:99
GET `/auth/sessions`	app/api/routes/auth.py:157	src/services/auth.ts:101
POST `/auth/sessions/revoke-others`	app/api/routes/auth.py:178	src/services/auth.ts:102
DELETE `/auth/sessions/{session_id}`	app/api/routes/auth.py:193	src/services/auth.ts:103
POST `/auth/email/send-verification`	app/api/routes/auth.py:217	src/services/auth.ts:105
POST `/auth/email/verify`	app/api/routes/auth.py:228	src/services/auth.ts:106
POST `/auth/password/forgot`	app/api/routes/auth.py:237	src/services/auth.ts:108
POST `/auth/password/reset`	app/api/routes/auth.py:248	src/services/auth.ts:110
PATCH `/auth/consent`	app/api/routes/auth.py:254	no screen uses it yet
GET `/onboarding/states`	app/api/routes/onboarding.py:27	src/services/onboarding.ts:35
GET `/onboarding/districts`	app/api/routes/onboarding.py:32	src/services/onboarding.ts:38
GET `/onboarding/facilities`	app/api/routes/onboarding.py:37	src/services/onboarding.ts:41
POST `/onboarding/profile`	app/api/routes/onboarding.py:49	src/services/onboarding.ts:43
GET `/onboarding/profile`	app/api/routes/onboarding.py:87	no screen uses it yet
PATCH `/onboarding/profile`	app/api/routes/onboarding.py:95	no screen uses it yet
GET `/health/metrics`	app/api/routes/vitals.py:59	src/services/health.ts:186
GET `/health/watch`	app/api/routes/vitals.py:138	src/services/health.ts:205
POST `/health/watch/connect`	app/api/routes/vitals.py:143	src/services/health.ts:206
POST `/health/watch/disconnect`	app/api/routes/vitals.py:153	src/services/health.ts:207
POST `/health/watch/sync`	app/api/routes/vitals.py:161	src/services/health.ts:209
GET `/health/baseline`	app/api/routes/vitals.py:240	src/services/health.ts:222
GET `/health/history`	app/api/routes/vitals.py:273	src/services/health.ts:213
GET `/health/summary`	app/api/routes/vitals.py:311	src/services/health.ts:219

Endpoint	Handler	Called from
DELETE `/health/readings/{reading_id}`	app/api/routes/vitals.py:402	src/services/health.ts:224
GET `/alerts`	app/api/routes/alerts.py:30	src/services/health.ts:227
GET `/alerts/unread-count`	app/api/routes/alerts.py:57	src/services/health.ts:230
POST `/alerts/mark-read`	app/api/routes/alerts.py:62	src/services/health.ts:232
GET `/schedule/mine`	app/api/routes/schedule.py:49	src/services/health.ts:235
POST `/schedule/mine/{visit_id}/reason`	app/api/routes/schedule.py:212	src/services/health.ts:237
POST `/schedule/mine/{visit_id}/cancel`	app/api/routes/schedule.py:235	src/services/health.ts:236
GET `/settings/preferences`	app/api/routes/settings.py:48	src/services/account.ts:87
PATCH `/settings/preferences`	app/api/routes/settings.py:53	src/services/account.ts:88
GET `/settings/consents`	app/api/routes/settings.py:86	src/services/account.ts:90
PATCH `/settings/consents`	app/api/routes/settings.py:91	src/services/account.ts:92
GET `/settings/consents/history`	app/api/routes/settings.py:104	src/services/account.ts:94
GET `/settings/about`	app/api/routes/settings.py:141	src/services/account.ts:96
GET `/settings/export`	app/api/routes/settings.py:187	src/services/account.ts:99
POST `/settings/password`	app/api/routes/settings.py:279	src/services/account.ts:109
POST `/settings/deactivate`	app/api/routes/settings.py:305	src/services/account.ts:111
POST `/settings/concerns`	app/api/routes/settings.py:349	src/services/account.ts:115
GET `/settings/concerns`	app/api/routes/settings.py:363	src/services/account.ts:113
GET `/settings/concerns/{concern_id}`	app/api/routes/settings.py:375	no screen uses it yet
GET `/settings/health-centre`	app/api/routes/settings.py:436	src/services/account.ts:117
PATCH `/settings/health-centre`	app/api/routes/settings.py:443	src/services/account.ts:118
GET `/profile`	app/api/routes/profile.py:116	src/services/account.ts:120
PATCH `/profile`	app/api/routes/profile.py:121	src/services/account.ts:122
POST `/profile/avatar`	app/api/routes/profile.py:174	src/services/account.ts:123
DELETE `/profile/avatar`	app/api/routes/profile.py:183	src/services/account.ts:128
POST `/profile/cover`	app/api/routes/profile.py:191	src/services/account.ts:123
DELETE `/profile/cover`	app/api/routes/profile.py:200	src/services/account.ts:128
POST `/profile/email`	app/api/routes/profile.py:217	src/services/account.ts:130
POST `/profile/email/confirm`	app/api/routes/profile.py:243	src/services/account.ts:131
GET `/assistant/info`	app/api/routes/assistant.py:86	src/services/assistant.ts:41
GET `/assistant/sessions`	app/api/routes/assistant.py:96	src/services/assistant.ts:42

Endpoint	Handler	Called from
GET `/assistant/sessions/{session_id}`	app/api/routes/assistant.py:108	src/services/assistant.ts:44
PATCH `/assistant/sessions/{session_id}`	app/api/routes/assistant.py:124	src/services/assistant.ts:45
DELETE `/assistant/sessions/{session_id}`	app/api/routes/assistant.py:139	src/services/assistant.ts:46
DELETE `/assistant/sessions`	app/api/routes/assistant.py:148	no screen uses it yet
POST `/assistant/ask`	app/api/routes/assistant.py:154	src/services/assistant.ts:49

Handlers link to the first line of the function; the third column links to the line that makes the request.

16 Data model cheat-sheet

The tables the citizen module reads and writes, with the columns worth knowing.

Table (model)	Key columns	Defined at
`users`	role, is_verified, is_active, profile_completed, consent, phc_id, district, state, pincode, area, geo_lat/lng, preferences (JSON), pending_email, deactivated_at	app/models/user.py:35
`user_profiles`	age, gender, height, weight, blood_group, conditions, medications, has_smartwatch, smartwatch_brand, emergency contact; one per user	app/models/user_profile.py:15
`health_records`	vitals (JSON), symptoms, timestamp (measured) vs created_at (stored), source (watch / phc / self), recorded_by, location, watch_day, watch_slot	app/models/health_record.py:14
`processed_data`	features (per-vital contribution), anomaly_score, is_anomaly, basis, details (audit), population, resolution, reviewed_by, reviewed_at; one per reading	app/models/processed_data.py:15
`alerts`	user_id, alert_type (personal / community), severity (low, medium, high), message, is_read	app/models/alert.py:10
`scheduled_visits`	worker_id, citizen_id, scheduled_for, status (scheduled, completed, cancelled, missed), visit_type, duration, reason, location, notes, cancelled_by, cancel_reason, missed_reason	app/models/scheduled_visit.py:11
`consent_records`	consent_type, granted, source, ip_address, created_at; append-only; latest per type wins	app/models/consent_record.py:11
`refresh_tokens`	token_hash (SHA-256), expires_at, revoked, revoked_reason, last_activity_at, device_label, rotated_from	app/models/refresh_token.py:10
`concerns`	ticket_ref, category, description, status, admin_note, subject_id, evidence (JSON)	app/models/concern.py:17
`advisories`	scope, district, area, radius_km, severity, body, expires_at, withdrawn_at, feedback	app/models/advisory.py:11
`valid_phc_facilities`	the registry: name, code, district, state, latitude, longitude, is_active	app/models/valid_phc_facility.py:12
`chat_sessions`, `chat_messages`	Nira conversations; each answer stores its retrieval sources and whether it was grounded	app/models/chat_message.py:15
`audit_logs`	who did what to which row from which IP (delete reading, change centre, deactivate)	app/models/audit_log.py:10

> `created_at` and `timestamp` differ on purpose: **timestamp is when the vital was measured, created_at is when it was stored**. All queries about 'when the reading happened' use `timestamp`.

17 Numbers to know

Every threshold the examiner might ask about, with the line that defines it.

Setting	Value	Defined at
Access token lifetime	15 minutes	app/core/config.py:14
Refresh token lifetime	30 days	app/core/config.py:15
Refresh idle timeout	48 hours	app/core/security.py:14
Refresh reuse grace (two tabs)	10 seconds	app/core/security.py:15
Email verification link	24 hours	app/core/security.py:16
Password reset link	1 hour	app/core/security.py:17
Resend an email	60 seconds apart	app/core/security.py:18
Password length	8 characters minimum, 72 bytes maximum	app/schemas/auth.py:34
Readings before a personal baseline starts	7	app/services/scoring.py:12
Readings for a fully personal baseline	21	app/services/scoring.py:13
Baseline looks at the latest	30 readings of each vital	app/services/scoring.py:14
A vital counts toward a flag from	contribution 0.5	app/services/scoring.py:28
Contribution that equals score 1.0	4.0	app/services/scoring.py:30
Earlier readings the scorer sees	120	app/services/readings.py:22
Severity bands	critical 0.75, high 0.5, moderate 0.25	app/services/triage.py:17
Dashboard windows	baseline 90 days, trend 30 days, up to 500 readings	app/api/routes/vitals.py:53
History page size	20 readings	src/pages/citizen/History.tsx:15
Badge refresh	every 30 seconds	src/components/app/AppShell.tsx:47
Visit grace before 'no outcome'	4 hours	app/services/visit_state.py:23
Cancel or move a visit	not within 8 hours; reason of 10+ characters	app/api/routes/schedule.py:30
Complaint about missed visits	3 unexplained in 90 days	app/services/complaints.py:22
Reading reminder	after 3 days quiet, hourly check, sent 09:00 to 20:00	app/services/reminders.py:27
Nira rate limit	20 questions a minute	app/core/config.py:33
Image upload limits	avatar 2 MB, cover 5 MB; PNG, JPEG, WebP	app/api/routes/profile.py:30

18 Questions across the whole module

Short answers with the place to open if the examiner says 'show me'.

Questions you may be asked

Q. Walk me through what happens when a citizen presses 'Fetch from watch'.
Chapter 5 table, then chapter 6. In one breath: page calls `syncWatch` → route checks role, consent and the slot → `simulate` → `submit_reading` stores and scores → flagged readings create an alert and queue emails → the page shows the result and the badge poll shows the alert.

Q. Where is the business logic and where is only plumbing?
Routes are plumbing (auth, validation, call a service). The rules are in `app/services`: `scoring.py`, `readings.py`, `consent.py`, `visits.py`, `complaints.py`, `preferences.py`.

Q. How do you make sure one citizen cannot see another's data?
Every citizen query filters on `user.id` taken from the verified token, never from the request. Examples: `delete_reading`, `mark_read`, `_own_session`. A foreign id answers 'not found', the same as an unknown id.

Q. What validates input?
Pydantic models on every route (types, lengths, patterns) plus the same rules mirrored in the forms. Vitals are range-checked by `validate_vitals` in the request schema before a worker's examination is saved, and the watch simulator clamps its values to plausible limits.

Q. What is stored about consent and why?
Each change is a new `consent_records` row with time, source and IP, so we can show what a person agreed to and when; the latest row is the current choice.

Q. How are emails prevented from being sent for a failed request?
They are queued on the session and sent in a hook that runs after a successful commit; a rollback drops them (`_send_after_commit`, `_drop_after_rollback`).

Q. What happens if the model or email fails?
The population model is optional (no model file means no population check). Email failures are logged with the address masked and do not fail the request.

Q. How would you replace the simulated watch?
Implement a function with the signature of `simulate` that calls the vendor API and returns the same four vitals; nothing else changes.

Q. What are the limits of the scoring?
It flags, it does not diagnose. The personal baseline needs about a week of data; until then only published ranges apply. Settings → Privacy and the flag explanations say so, and an emergency keyword gets 'call 108' immediately in Nira.

Q. How is the app protected against someone stealing a session?
Short access tokens, rotating refresh tokens with reuse detection, httpOnly cookie, idle timeout, a sessions page to sign out devices, and a password change signs out every other session.

Q. What would you test first?
Authorisation (foreign ids), the once-per-slot rule, consent effects, the 8-hour cancellation, scoring edge cases (fewer than 7 readings, safety limits, the confirmation rule). The test files in `backend/tests` are named by feature: test_auth, test_onboarding, test_watch, test_scoring, test_consents, test_notifications, test_modules.